

1003 L298N: Electronics Lab Experiment Plan

STEM Starter Kit Series, Part 4, supporting lab outline (planning draft)

Prepared for Jesse Carpenter, Carpenter Software · August 29, 2026 · Article 1003 text is not modified by this document.

Plan and Design Rationale

This document lays out ten experiments (labs) built entirely from the six items labeled in the Electronics Lab photograph: the Arduino Uno, the standalone ATmega328P, the L298N motor driver, the LS7366R quadrature counter, the Cytron 12VDC geared motors with encoders, and the thumb joystick. It expands the four example labs already sketched, I2C wiring, MCU-to-L298N wiring, joystick wiring, and joystick programming, into a full build sequence that touches every component in the photograph. Nothing in Article 1003 has been changed; this is a planning outline only, meant to be checked against your other sources before it is written into the article.

Architecture assumptions

Two assumptions set the wiring direction in every experiment below and should be confirmed against your other sources before the labs are drafted in full.

- 1 The Arduino Uno is the I2C master and the joystick input device. It reads the thumb joystick and sends motion commands over the I2C bus.
- 2 The standalone ATmega328P is the I2C slave and does two jobs at once, as you described: it drives the L298N motor outputs, and it reads the two LS7366R quadrature counters over SPI, one counter per motor.

If either assumption does not match your other sources, the wiring steps in Experiments 1, 2, 4, 8, and 9 would need to be mirrored between the two boards.

Sequencing principle

The order follows a principle already stated in the article's own introduction: several labs need no motor at all, so a beginner can verify each piece before ever connecting one. Experiments 1, 6, and 7 test the I2C link and the joystick with no motor present. Experiments 2 and 3 introduce the L298N, first with one motor, then with both. Experiments 4 and 5 add the encoders and counters, which can be verified by turning a motor shaft by hand rather than driving it electrically. Experiments 8 through 10 combine every subsystem into the finished joystick-controlled drivetrain.

Where your four example labs landed

Your label	Description	Final experiment
Lab-1	Wiring the Uno to the ATmega328P, programming the I2C	Experiment 1
Lab-2	Wiring the MCU to the L298N	Experiments 2 and 3 (split: one motor wired and tested first, then a second motor added for independent two-channel control)
Lab-3	Wiring the thumb joystick to the Uno	Experiment 6
Lab-4	Programming the Uno and joystick (Article 1009 Supplemental)	Experiment 7

Electronics Lab component coverage

#	Component	Used in experiments
1	Arduino Uno	1, 6, 7, 8, 9, 10
2	ATmega328P (MCU)	1, 2, 3, 4, 5, 8, 9, 10
3	L298N motor driver	2, 3, 8, 9, 10
4	LS7366R quadrature counter (x2)	4, 5, 9, 10
5	Cytron 12VDC geared motors with encoders	2, 3, 4, 5, 8, 9, 10
6	Thumb joystick	6, 7, 8, 10

Experiment Index

#	Title	Description
1	I2C Link: Uno to ATmega328P	Wire the two boards over I2C and verify the link with a byte-level ping and echo test. No motor involved.
2	L298N Single-Motor Wiring Test	Wire the MCU to the L298N and drive one Cytron motor forward, reverse, and stop.
3	Two-Motor PWM Speed and Direction	Add the second motor and verify independent speed and direction control on both channels.
4	Wiring the Encoders to the LS7366R	Wire each motor's quadrature encoder output to its own LS7366R counter chip.
5	Programming the LS7366R Counters	Initialize both counters over SPI and read live counts by turning each motor shaft by hand.
6	Wiring the Thumb Joystick to the Uno	Wire the joystick's two axes and its switch to the Uno. No motor involved.
7	Reading the Joystick (Article 1009)	Read raw axis values, apply dead zone and interpolation, and print a speed command.
8	I2C Integration: Joystick to Motors	Send joystick-derived commands from the Uno to the MCU over I2C and confirm both motors respond.
9	Telemetry: Counters Back Over I2C	Add counter reads to the same MCU firmware so the Uno can request and display wheel counts.
10	Full System Bench Test	Run the complete assembly through a motion test matrix and a troubleshooting checklist.

Experiment Details

Experiment 1: I2C Link: Uno to ATmega328P

Objective. Establish the physical I2C connection between the Uno and the standalone ATmega328P, and confirm the link works with a minimal ping and echo test before any motor hardware is involved.

Builds on. Nothing (first experiment).

Electronics Lab components. 1. Arduino Uno, 2. ATmega328P (breadboard circuit with its own power, oscillator, and reset wiring already in place).

Steps

- 1 Confirm the ATmega328P breadboard circuit has power, a working clock source, and a pulled-up reset line before connecting anything else.
- 2 Wire SDA and SCL between the Uno and the MCU, and tie the two grounds together.
- 3 Add pull-up resistors on SDA and SCL if the boards do not already supply them.
- 4 Program the Uno as the I2C master, sending one test byte on a fixed interval.
- 5 Program the MCU as the I2C slave, receiving the byte and returning a fixed echo or status byte.
- 6 Read the echoed byte on the Uno's Serial Monitor to confirm two-way communication.

Verify. The Serial Monitor shows the expected echo byte on every request, with no missed or garbled transfers over a short observation period.

Watch for. Address conflicts between master and slave, a missing pull-up, a floating MCU reset pin, and a bus speed mismatch are the most common first-lab failures.

Experiment 2: L298N Single-Motor Wiring Test

Objective. Wire the MCU's output pins to the L298N's control inputs and confirm basic direction and speed control on one motor channel, independent of the I2C link built in Experiment 1.

Builds on. Experiment 1 (MCU is already powered and programmable).

Electronics Lab components. 2. ATmega328P, 3. L298N motor driver, 5. one Cytron 12VDC geared motor, plus a 12V supply.

Steps

- 1 Identify the L298N's IN1, IN2, and ENA pins for the first channel (consult the L298N datasheet or breakout board silkscreen for exact labeling).
- 2 Wire two MCU digital pins to IN1 and IN2 for direction, and one PWM-capable pin to ENA for speed.
- 3 Wire the first Cytron motor's power leads to the L298N's OUT1 and OUT2 terminals.
- 4 Connect the 12V motor supply to the L298N's motor power input, keeping it separate from the 5V logic supply, and confirm the logic and motor grounds are common.
- 5 Program the MCU to run the motor forward, reverse, and stop, then vary the PWM duty cycle to confirm speed control.

Verify. The motor spins in both directions on command and visibly slows as duty cycle is reduced.

Watch for. Reversed motor leads (direction is backward), a missing common ground between logic and motor supply, and an L298N running hot without a heat sink.

Experiment 3: Two-Motor PWM Speed and Direction

Objective. Extend Experiment 2 to the second Cytron motor on the L298N's other H-bridge channel, and drive both motors independently.

Builds on. Experiment 2.

Electronics Lab components. 2. ATmega328P, 3. L298N motor driver, 5. both Cytron 12VDC geared motors.

Steps

- 1 Wire the second motor to the L298N's OUT3 and OUT4 terminals.
- 2 Wire IN3, IN4, and ENB from the MCU for the second channel, following the same pattern as channel one.
- 3 Program the MCU to run simple two-motor patterns: both forward, both reverse, one forward and one reverse (pivot), and independent speeds on each side.
- 4 Confirm the H-bridge truth table for both channels matches the observed motor behavior.

Verify. Both motors respond independently and correctly to the programmed direction and speed patterns.

Watch for. Crosstalk between channels if grounds are not solid, and a motor supply that browns out when both motors draw current at once.

Experiment 4: Wiring the Encoders to the LS7366R

Objective. Wire each Cytron motor's quadrature encoder output to its own LS7366R counter chip, and wire both counters to the MCU's SPI bus.

Builds on. Experiment 1 (MCU and its power/SPI wiring pattern). Does not require Experiments 2 or 3.

Electronics Lab components. 2. ATmega328P, 4. LS7366R (x2), 5. both Cytron motors (encoder leads only, no motor drive power required for this experiment).

Steps

- 1 Identify the encoder leads on each Cytron motor cable: encoder power, ground, channel A, and channel B (consult the Cytron SPG30 datasheet for the exact wire colors and pinout).
- 2 Wire each motor's channel A and channel B outputs to its own LS7366R's quadrature inputs.
- 3 Wire both LS7366R chips to the MCU's SPI bus (shared MOSI, MISO, and SCK), giving each chip its own chip-select line so the MCU can address them individually.
- 4 Add any decoupling capacitors called for in the LS7366R datasheet.

Verify. Both LS7366R chips power up cleanly and respond to an SPI read of a status or configuration register (checked in Experiment 5).

Watch for. Swapped channel A and B leads (counts run backward), and two counters sharing a chip-select line by mistake.

Experiment 5: Programming the LS7366R Counters

Objective. Initialize both LS7366R chips over SPI and read live quadrature counts as each motor shaft is turned by hand, with no motor drive power needed.

Builds on. Experiment 4.

Electronics Lab components. 2. ATmega328P, 4. LS7366R (x2), 5. both Cytron motors (shaft turned by hand).

Steps

- 1 Set up the MCU's SPI peripheral as master.
- 2 Write the LS7366R's mode registers to select quadrature x4 counting.
- 3 Read each chip's count register over SPI in a loop, selecting one chip at a time with its chip-select line.
- 4 Turn each motor shaft by hand in both directions and confirm the corresponding count increases or decreases.
- 5 For this experiment only, report counts directly from the MCU (a temporary USB-serial connection or a debug LED pattern is enough); full-time reporting through the Uno is added in Experiment 9.

Verify. Turning a shaft by hand produces a matching, direction-correct change in that channel's count, with the other channel's count unaffected.

Watch for. A count that does not move (wiring or mode-register error), and a count that changes on both chips at once (chip-select wiring error).

Experiment 6: Wiring the Thumb Joystick to the Uno

Objective. Physically connect the joystick breakout board to the Uno. No motor or MCU involved.

Builds on. Nothing (can be built in parallel with Experiments 1 through 5).

Electronics Lab components. 1. Arduino Uno, 6. thumb joystick.

Steps

- 1 Wire the joystick's VRx and VRy analog outputs to two Uno analog input pins.
- 2 Wire the joystick's push-switch output to a Uno digital pin, using the Uno's internal pull-up.
- 3 Wire the joystick's power and ground to the Uno's 5V and ground.
- 4 Mount or position the joystick within easy reach on the bench, as shown in the Electronics Lab photograph.

Verify. A multimeter check across VRx and VRy shows a voltage that changes smoothly as the stick is moved, centering near half the supply voltage at rest.

Watch for. A floating or noisy reading if ground is not connected, and a switch that reads the wrong logic level if the pull-up is not enabled.

Experiment 7: Reading the Joystick (Article 1009)

Objective. Read raw analog values from both joystick axes and the switch, then apply the dead-zone and interpolation technique from Article 1009 (L298N Supplemental, Joystick Setup) to produce a usable speed command.

Builds on. Experiment 6.

Electronics Lab components. 1. Arduino Uno, 6. thumb joystick.

Steps

- 1 Read both axes with `analogRead` and the switch with `digitalRead`.
- 2 Record the at-rest center value for each axis and apply a dead zone around it, per Article 1009's method.
- 3 Map the remaining range on each axis to a signed speed value using the interpolation approach from Articles 1001 and 1009.
- 4 Print the resulting speed values to the Serial Monitor and confirm they read zero at rest and scale smoothly toward full deflection.

Verify. Speed values sit at zero within the dead zone and move smoothly and symmetrically as the stick is pushed toward each extreme.

Watch for. A dead zone set too narrow (motor jitters near center once connected later) or too wide (a large dead area with no response).

Experiment 8: I2C Integration: Joystick to Motors

Objective. Combine Experiment 1 (I2C link), Experiment 3 (two-motor PWM control), and Experiment 7 (joystick reading) so the Uno sends joystick-derived commands to the MCU, which drives both motors accordingly.

Builds on. Experiments 1, 3, and 7.

Electronics Lab components. 1. Arduino Uno, 2. ATmega328P, 3. L298N motor driver, 5. both Cytron motors, 6. thumb joystick.

Steps

- 1 Define a simple command packet, for example a signed left-speed byte and a signed right-speed byte, sent from Uno to MCU.
- 2 On the Uno, package the joystick's interpolated speed values from Experiment 7 and transmit them over I2C on a fixed interval.
- 3 On the MCU, receive the packet in the I2C slave handler and reuse the Experiment 3 logic to set L298N direction and PWM duty cycle for both motors.
- 4 Bench test by moving the joystick and observing that both motors respond with the expected direction and speed.

Verify. Motor response tracks joystick position in real time, with both channels responding independently to combined X and Y motion.

Watch for. A visible lag or dropped commands if the I2C update rate is too slow, and one motor not responding if the command packet's byte order does not match on both ends.

Experiment 9: Telemetry: Counters Back Over I2C

Objective. Add the LS7366R counter reads from Experiment 5 into the same MCU firmware used in Experiment 8, so the MCU can report wheel counts back to the Uno on request.

Builds on. Experiments 5 and 8.

Electronics Lab components. 1. Arduino Uno, 2. ATmega328P, 4. LS7366R (x2), 5. both Cytron motors.

Steps

- 1 Extend the MCU's I2C slave handler to recognize a request byte from the Uno asking for current counts.
- 2 On that request, read both LS7366R count registers over SPI (Experiment 5 logic) and return them to the Uno.
- 3 On the Uno, request counts periodically and print left and right values to the Serial Monitor.
- 4 Optionally compute an approximate wheel speed from the change in count over a known time interval, and display it alongside the raw counts.

Verify. Counts reported over I2C match the counts observed directly in Experiment 5 for the same shaft motion, and update consistently as the motors run under joystick control.

Watch for. Counts that reset or skip if the request and the SPI read happen in the wrong order relative to the motor PWM interrupt (if one is used).

Experiment 10: Full System Bench Test

Objective. Run the complete Electronics Lab assembly as a bench-mounted, joystick-driven two-wheel motion demonstrator, and verify its behavior against a fixed test matrix.

Builds on. Experiments 1 through 9 (all prior experiments).

Electronics Lab components. 1. Arduino Uno, 2. ATmega328P, 3. L298N motor driver, 4. LS7366R (x2), 5. both Cytron motors, 6. thumb joystick.

Steps

- 1 Power up in sequence: logic power first, then motor power, checking common grounds with a multimeter before applying motor power, in keeping with the measurement-first approach from Article 1000.
- 2 Run a fixed test matrix: forward, reverse, left pivot, right pivot, center and stop, and a diagonal blend of X and Y.
- 3 For each entry in the matrix, record the commanded joystick position, the observed motor behavior, and the reported encoder counts.
- 4 Work through a troubleshooting table for common faults: no I2C acknowledgment, one motor not responding, joystick drift at rest, and a counter that reads zero.

Verify. Every entry in the test matrix produces the expected motion, and the recorded encoder counts are consistent with the direction and duration of each motion.

Watch for. This experiment is a system-level check, not a new subsystem; most faults found here trace back to a specific earlier experiment and should be re-verified there.

End of experiment plan. Ready for review against your other sources before Article 1003 is drafted.